

Universidade Federal de Ouro Preto - UFOP
Instituto de Ciências Exatas e Biológicas - ICEB
Departamento de Computação - DECOM
Ciência da Computação

MiniCCO

Núcleo Geométrico e Estrutural para Crescimento Arterial Simplificado

Ana Paula dos Santos Tadim, Heitor Augusto de Oliveira Ramos, Matheus Maia
Amorim

Professor: Rafael Alves Bonfim de Queiroz

Ouro Preto
13 de julho de 2026

Sumário

1	Introdução	1
1.1	Considerações iniciais	1
1.2	Instruções de compilação e execução	1
2	Descrição da implementação	2
2.1	Organização dos Arquivos	2
3	Exemplos de execução	4
4	Análise de Resultados	7
4.1	Análise do Relatório Final e a Equação de Nós	7
4.2	Cenário 1: Simulação com 10 Terminais, $\gamma = 3.0$, $M=10$	8
4.3	Cenário 2: Simulação com 30 Terminais, $(\gamma) = 3.0$, $(M)=20$	9
4.4	Cenário 3: Simulação com 50 Terminais, $(\gamma) = 3.0$, $(M)=20$	9
4.5	Cenário 4: Simulação com 50 Terminais, $(\gamma) = 2.7$, $(M)=20$	9
4.6	Cenário 5: Simulação com 50 Terminais, $(\gamma) = 3$, $(M)=50$	10
5	Considerações Finais	12

Lista de Figuras

1	./minicco1 10 10.0 3.0 10	4
2	./minicco1 30 10.0 3.0 20	5
3	./minicco1 50 10.0 3.0 20	5
4	./minicco1 50 10.0 2.7 20	6
5	./minicco1 50 10.0 3.0 50	6
6	Árvores geradas para diferentes quantidades de terminais.	7

Lista de Tabelas

1	Relatório da Árvore	8
2	Relatório da Árvore	9
3	Relatório da Árvore.	9
4	Relatório da Árvore	9
5	Relatório da Árvore	10

1 Introdução

As árvores arteriais têm a função de transportar sangue aos tecidos, garantindo o suprimento de oxigênio e nutrientes necessários para o metabolismo celular. A complexidade da estrutura vascular observada na natureza levanta uma importante questão científica: por que os vasos sanguíneos apresentam determinados padrões de crescimento, ramificação e bifurcação? Estudos experimentais realizados em árvores arteriais humanas mostraram que as bifurcações seguem relações geométricas específicas envolvendo raios dos vasos, ângulos de ramificação e distribuição do fluxo sanguíneo. Essas características sugerem a existência de princípios de otimização capazes de explicar a organização da rede vascular.

O algoritmo CCO (Constrained Constructive Optimization) é um método utilizado para a geração de árvores arteriais artificiais. Seu objetivo é construir uma rede de vasos sanguíneos capaz de irrigar uma determinada região do domínio, respeitando restrições geométricas e fisiológicas. A ideia principal consiste em adicionar novos terminais à árvore de forma incremental. Para cada novo ponto gerado, o algoritmo busca a melhor conexão possível com a estrutura existente, criando bifurcações e novos segmentos. Durante esse processo, critérios de otimização são utilizados para obter uma árvore organizada e com características semelhantes às observadas em sistemas vasculares reais.

1.1 Considerações iniciais

Para a criação desse projeto, foram utilizadas algumas ferramentas de desenvolvimento, como:

- Ambiente de desenvolvimento do código fonte: **Visual Studio Code**.¹
- Linguagem utilizada: **C++**.
- Ambiente de desenvolvimento da documentação: **Overleaf L^AT_EX**.²

1.2 Instruções de compilação e execução

Para a compilação do projeto, foi utilizado um arquivo **Makefile**, com o objetivo de simplificar o processo de construção do executável. O arquivo contém as instruções necessárias para compilar todos os arquivos-fonte da aplicação e gerar o programa final.

Compilando o projeto

```
make
```

O comando acima executa as instruções definidas no arquivo *Makefile*, equivalente à compilação manual por meio do comando:

Compilação manual

```
g++ -I ../include -std=c++11 domain.cpp geometry.cpp main.cpp random.cpp tree.cpp -o minicco1
```

As diretivas utilizadas na compilação são:

- *g++*: compilador C++ utilizado no desenvolvimento do projeto;
- *-I ../include*: indica ao compilador o diretório onde se encontram os arquivos de cabeçalho (.hpp);
- *-std=c++11*: Define o padrão da linguagem C++11 para a compilação do código.
- *domain.cpp ... tree.cpp*:: Lista explicitamente os arquivos-fonte que serão compilados.
- *-o minicco1*: Especifica o nome do arquivo executável final que será gerado.

¹Visual Studio Code está disponível em <https://code.visualstudio.com/>

²Disponível em <https://www.overleaf.com/>

Após a compilação, a execução da simulação é realizada informando a quantidade de terminais desejada e o raio do domínio circular:

Execução do programa

```
./ minicco1 Nterm R gamma M
```

Por exemplo:

Exemplo de execução

```
./ minicco1 50 0.05 3.0 20
```

Neste exemplo, será gerada uma árvore arterial contendo 10 terminais em um domínio circular de raio igual a 10 unidades.

Após a execução, são gerados os arquivos `arvore.vtk` e `raio.txt`, utilizados na etapa de visualização da árvore. A visualização gráfica é realizada através do script Python:

Visualização da árvore

```
python3 visualizar.py arvore.vtk
```

O comando abre uma janela gráfica contendo a árvore arterial gerada, os segmentos da estrutura vascular e o domínio circular utilizado durante a simulação.

2 Descrição da implementação

O modelo inicial simplificado inspirado no método CCO (Constrained Constructive Optimization), foi desenvolvido em C++ para a geração da árvore arterial e em Python para a visualização gráfica dos resultados. A estrutura do código foi organizada em duas pastas principais:

- **include/**: contém os arquivos de cabeçalho (.hpp), responsáveis pelas definições das estruturas de dados e protótipos das funções do sistema.
- **src/**: contém os arquivos de implementação (.cpp), o arquivo Makefile e o script de visualização.

2.1 Organização dos Arquivos

A pasta include contém os seguintes arquivos:

- **point.hpp**: definição da estrutura de pontos bidimensionais.
- **segment.hpp**: representação dos segmentos da árvore arterial.
- **node.hpp**: estrutura dos nós utilizados na árvore.
- **candidate.hpp**: definição dos candidatos avaliados para conexão de novos terminais.
- **tree.hpp**: estrutura principal da árvore arterial e protótipos das operações de manipulação.
- **geometry.hpp**: funções geométricas utilizadas nos cálculos de distância e interseção.
- **domain.hpp**: funções relacionadas ao domínio circular da simulação.
- **random.hpp**: funções auxiliares para geração de valores aleatórios.

A pasta src contém:

- **main.cpp**: implementação principal do algoritmo de crescimento da árvore.
- **tree.cpp**: operações de inserção de nós, geração de candidatos e exportação da árvore.

- **geometry.cpp**: implementação dos cálculos geométricos.
- **domain.cpp**: geração de pontos válidos dentro do domínio.
- **random.cpp**: funções auxiliares para sorteio de coordenadas.
- **visualizar.py**: responsável pela visualização gráfica da árvore gerada.
- **Makefile**: automatiza o processo de compilação.

Inicialmente é criado um nó de entrada, denominado raiz da árvore, posicionado em:

$$(0, R)$$

Geração dos Terminais

O crescimento da árvore ocorre de forma iterativa. A cada iteração é gerado um novo terminal candidato dentro do domínio circular.

Diferentemente de abordagens baseadas em posições pré-definidas, os pontos não são inseridos utilizando coordenadas fixas. Cada novo terminal é obtido através de uma busca aleatória, sendo aceito somente quando satisfaz um conjunto de restrições geométricas.

Entre os critérios utilizados destacam-se:

- Distância mínima em relação aos terminais já existentes;
- Distância mínima em relação aos segmentos da árvore;
- Distância mínima em relação ao ponto de entrada da árvore;
- Respeito aos limites do domínio circular.

Para a geração do primeiro terminal também são aplicadas restrições adicionais para evitar que ele seja criado excessivamente próximo da entrada ou do centro geométrico do domínio.

Seleção da Melhor Conexão

Após a geração de um terminal válido, o algoritmo avalia *todos* os segmentos existentes na árvore como possíveis candidatos para conexão. Para cada segmento candidato, uma bifurcação é inserida temporariamente e sua posição é refinada por uma otimização geométrica local (ver seção seguinte): a árvore inteira é recalculada fisicamente (fluxos, raios pela Lei de Murray, resistências e volumes) e o custo do candidato é definido como o volume intravascular total resultante, $J = \sum_j \pi r_j^2 l_j$ – e não uma distância geométrica simples. Candidatos que produziram interseção com segmentos já existentes, que caem fora do domínio circular, ou que ficam próximos demais de outro nó já existente na árvore são descartados. Entre os candidatos geometricamente válidos, o segmento cuja bifurcação otimizada resulta no menor volume total é selecionado através de uma estratégia gulosa (*greedy*).

Inserção de Bifurcações

Quando um segmento é escolhido como candidato, ele não é dividido exatamente no seu ponto médio: o ponto médio é usado apenas como *ponto de partida* de uma busca em grade sobre coordenadas baricentrizadas do triângulo formado pelas extremidades do segmento e o novo terminal. Essa busca é restrita a uma vizinhança local de 30% ao redor do ponto médio – e não ao triângulo inteiro – de modo que a otimização geométrica permaneça *local*, como descrito no algoritmo CCO clássico. Dentro dessa vizinhança, o algoritmo escolhe o ponto de bifurcação X^* que minimiza o volume total da árvore, respeitando as restrições geométricas (domínio, não interseção, distância mínima entre nós). O segmento original é então substituído por três segmentos: o trecho entre o início original e o ponto de bifurcação X^* , o trecho entre X^* e o final original, e o novo ramo conectando X^* ao terminal recém-gerado. Esse processo é repetido até que a quantidade de terminais especificada pelo usuário seja atingida.

Exportação e Visualização

Ao final da execução, a árvore construída é exportada para um arquivo no formato VTK (*Visualization Toolkit*), permitindo sua visualização através do software desenvolvido em Python utilizando a biblioteca **PyVista**.

Na visualização:

- a raiz é exibida em laranja;
- os demais nós são exibidos em verde;
- os segmentos recebem cores alternadas para facilitar a identificação da estrutura;
- o domínio circular é representado por uma circunferência de contorno preto.

Dessa forma, é possível analisar visualmente o crescimento e a organização da árvore arterial gerada pelo algoritmo.

3 Exemplos de execução

Para avaliar o comportamento do algoritmo implementado, foram realizados experimentos utilizando um domínio circular de raio $R = 10$, variando a quantidade de terminais (N_{term}), parâmetros γ (expoente da relação entre fluxo e raio dos vasos) e M (define o número máximo de candidatos avaliados durante a busca pelo melhor segmento para conexão de um novo terminal).

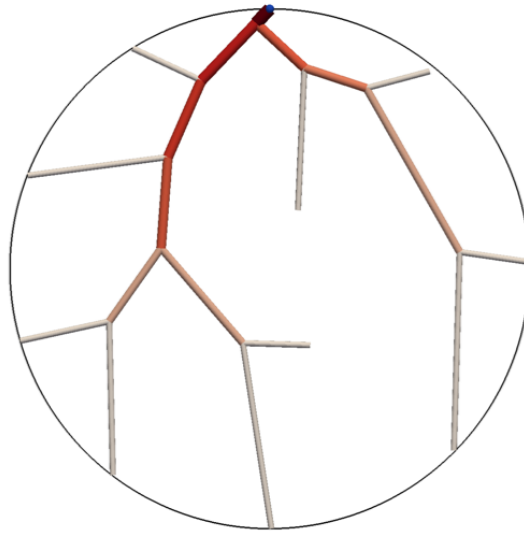


Figura 1: `./minicco1 10 10.0 3.0 10`

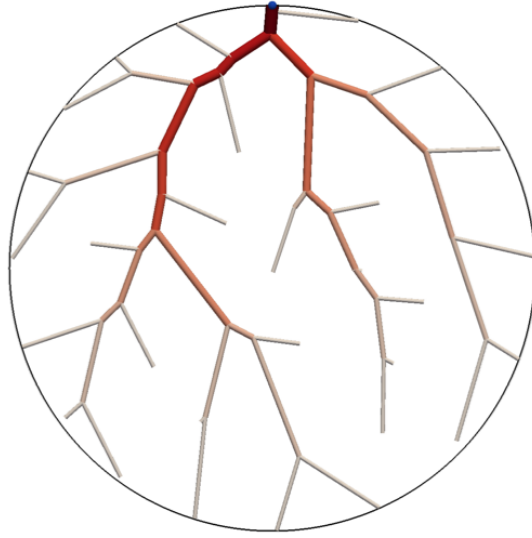


Figura 2: `./minicco1 30 10.0 3.0 20`

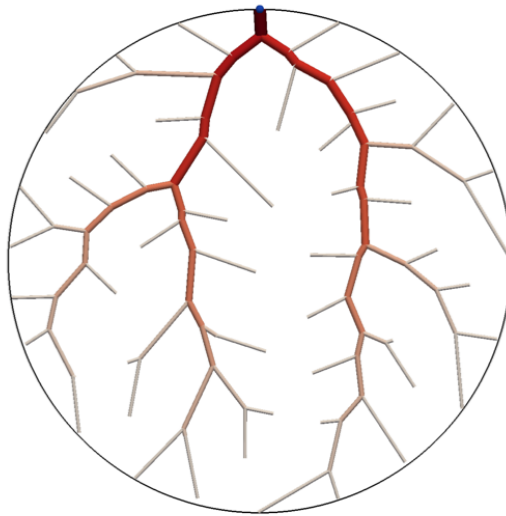


Figura 3: `./minicco1 50 10.0 3.0 20`

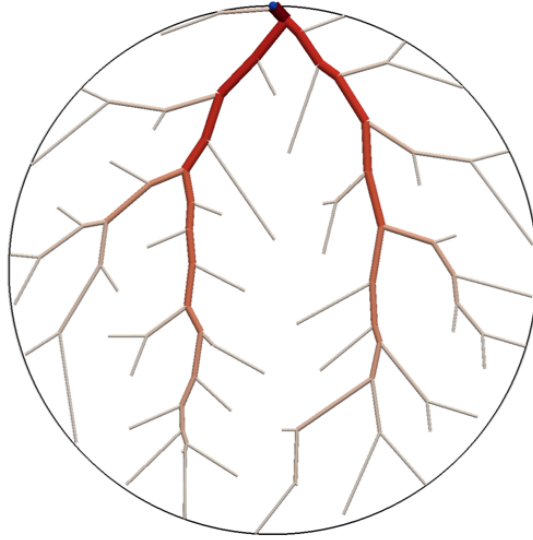


Figura 4: `./minicco1 50 10.0 2.7 20`

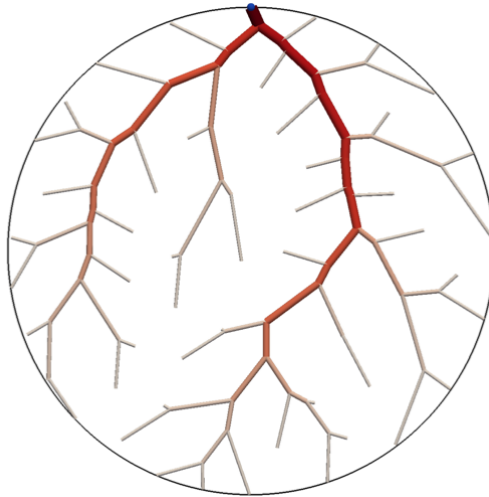
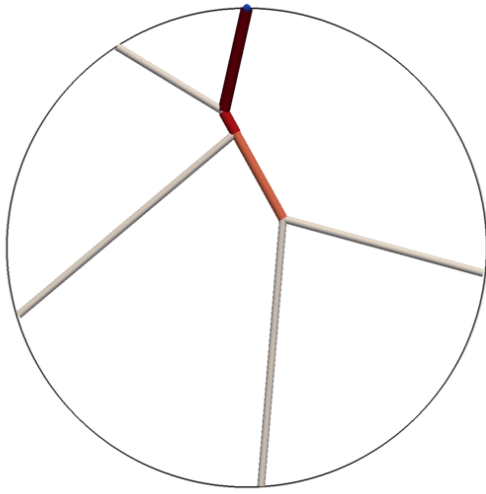
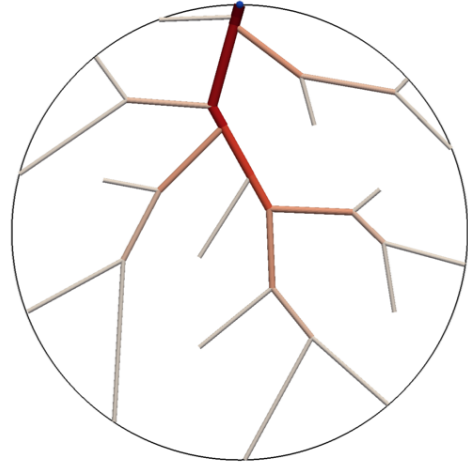


Figura 5: `./minicco1 50 10.0 3.0 50`

Outros exemplos com valores fixos de $\gamma = 3.0$ e $M = 5$

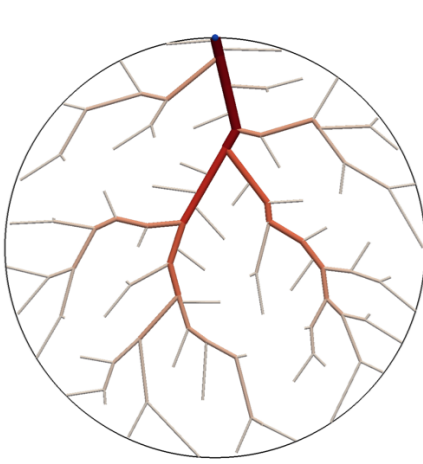


(a) $N_{\text{terminais}} = 4$

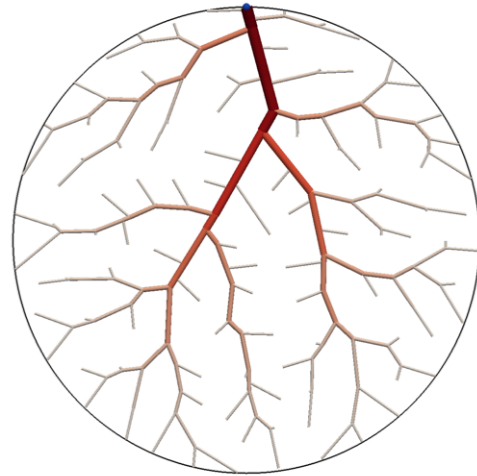


(b) $N_{\text{terminais}} = 16$

Figura 6: Árvores geradas para diferentes quantidades de terminais.



(a) $N_{\text{terminais}} = 60$



(b) $N_{\text{terminais}} = 100$

4 Análise de Resultados

A análise de resultados foi realizada por meio de testes progressivos com diferentes quantidades de terminais. A seguir, detalham-se as saídas com os seguintes testes:

4.1 Análise do Relatório Final e a Equação de Nós

Um aspecto fundamental observado em todas as execuções é a relação entre o número de terminais solicitados e o número total de nós gerados na árvore. Notavelmente, ao solicitar N terminais, o programa reporta exatamente $2N$ nós totais (por exemplo, 5 terminais geram 10 nós, e 10 terminais geram 20 nós). Essa propriedade é explicada pela topologia de uma árvore binária estrita gerada pelo modelo CCO. A estrutura é regida pela seguinte fórmula matemática:

$$\text{Total_de_Nos} = 2 \times N_{\text{folhas}}$$

A razão para isso ocorrer é que cada novo terminal aceito não apenas adiciona ele mesmo à árvore, mas também força a criação de um **nó interno de bifurcação** no segmento alvo onde ele se conectou.

A adição de cada folha subsequente equilibra a topologia criando exatamente um par de nós (1 nó interno + 1 nó terminal), fazendo com que o total sempre dobre. Essa mesma topologia binária é o que permite calcular o fluxo de cada segmento sem precisar resolver nenhum sistema de equações: como cada bifurcação apenas distribui o fluxo que já chega até ela, o fluxo Q_j de um segmento j é diretamente proporcional ao número de terminais que existem na sub-árvore abaixo dele. Ou seja, se um segmento tem 5 nós terminais em sua descendência, ele carrega exatamente o fluxo equivalente a 5 terminais, não é preciso somar fluxo bifurcação por bifurcação a partir da raiz, basta contar quantas folhas existem abaixo daquele segmento. Dessa forma, seja n_j o número de terminais distais ao segmento j (contado recursivamente: $n_j = 1$ se j é folha, e $n_j = n_{esq} + n_{dir}$ se j é uma bifurcação). Assumindo fluxo terminal igual Q_{term} para todos os terminais (distribuição uniforme de demanda), o fluxo em qualquer segmento é dado por

$$Q_j = n_j \times Q_{term}.$$

Isso é consequência direta da conservação de massa: o fluxo que entra em uma bifurcação é sempre igual à soma dos fluxos que saem dela, e como o fluxo só se origina nos terminais, basta contar quantos terminais existem "abaixo" de um segmento para saber seu fluxo. Isso é o que a função `atualizaQtdTerminaisDistais` calcula (contagem recursiva de n_j , em pós-ordem), e `atualizaFluxos` converte diretamente em Q_j multiplicando por Q_{term} . —

4.2 Cenário 1: Simulação com 10 Terminais, $\gamma = 3.0$, $M=10$

Para qualquer teste é imprimido os valores das coordenadas dos terminais inseridos, um relatório da árvore gerada, no .vtk, e uma tabela com o id de cada segmentos e os valores dos fluxos atualizados a cada nova inserção após a execução final.

Saída do programa: Terminal		
Terminal 1/10 inserido.	Coordenadas:	(-0.661685, -9.97801)
Terminal 2/10 inserido.	Coordenadas:	(6.06507, 7.9185)
Terminal 3/10 inserido.	Coordenadas:	(-9.8935, 0.750946)
Terminal 4/10 inserido.	Coordenadas:	(9.1887, -3.39594)
Terminal 5/10 inserido.	Coordenadas:	(-7.51014, -6.5672)
Terminal 6/10 inserido.	Coordenadas:	(-1.69987, -3.03953)
Terminal 7/10 inserido.	Coordenadas:	(-6.10832, 7.89585)
Terminal 8/10 inserido.	Coordenadas:	(5.25299, -8.31987)
Terminal 9/10 inserido.	Coordenadas:	(9.48839, 3.01654)
Terminal 10/10 inserido.	Coordenadas:	(-1.05024, 2.26351)

Tabela 1: Relatório da Árvore

Métrica	Valor
Número total de nós	20
Número total de segmentos	19
Número de terminais	10
Comprimento total	77.1842
Volume total	0.00130728
Raio da raiz	0.00409839
Raio médio dos segmentos	0.00241075
Conexões testadas	2660
Conexões rejeitadas	1606
Tempo de execução (s)	0.122686

4.3 Cenário 2: Simulação com 30 Terminais, $(\gamma) = 3.0$, $(M)=20$

Tabela 2: Relatório da Árvore

Métrica	Valor
Número total de nós	60
Número total de segmentos	59
Número de terminais	30
Comprimento total	134.852
Volume total	0.00141733
Raio da raiz	0.00409923
Raio médio dos segmentos	0.00183621
Conexões testadas	105111
Conexões rejeitadas	82213
Tempo de execução (s)	4.97884

4.4 Cenário 3: Simulação com 50 Terminais, $(\gamma) = 3.0$, $(M)=20$

Tabela 3: Relatório da Árvore.

Métrica	Valor
Número total de nós	100
Número total de segmentos	99
Número de terminais	50
Comprimento total	164.785
Volume total	0.00150893
Raio da raiz	0.00418932
Raio médio dos segmentos	0.00168551
Conexões testadas	300202
Conexões rejeitadas	257351
Tempo de execução (s)	23.1002

4.5 Cenário 4: Simulação com 50 Terminais, $(\gamma) = 2.7$, $(M)=20$

Tabela 4: Relatório da Árvore

Métrica	Valor
Número total de nós	100
Número total de segmentos	99
Número de terminais	50
Comprimento total	173.682
Volume total (J)	0.00151074
Raio da raiz	0.00452193
Raio médio dos segmentos	0.00161757
Conexões testadas	299972
Conexões rejeitadas	259500
Tempo de execução (s)	23.8954

4.6 Cenário 5: Simulação com 50 Terminais, $(\gamma) = 3$, $(M)=50$

Tabela 5: Relatório da Árvore

Métrica	Valor
Número total de nós	100
Número total de segmentos	99
Número de terminais	50
Comprimento total	167.910
Volume total (J)	0.00153630
Raio da raiz	0.00416888
Raio médio dos segmentos	0.00166371
Conexões testadas	1823936
Conexões rejeitadas	1546002
Tempo de execução (s)	145.813

Discussão sobre Complexidade

1. Como a escala dos raios altera a árvore gerada?

A função `normalizaRaiosPorResistenciaGlobal` multiplica o raio de todos os segmentos pelo mesmo fator

$$k = \left(\frac{R_{\text{bruto}}}{R_{\text{alvo}}} \right)^{1/4},$$

em que R_{bruto} representa a resistência hidráulica equivalente da árvore antes da normalização e $R_{\text{alvo}} = \Delta P / Q_{\text{perf}}$ corresponde à resistência desejada. Como o fator de escala é aplicado igualmente a todos os segmentos, essa operação não altera a topologia nem a geometria da árvore, ou seja, as posições dos nós, os comprimentos e os ângulos permanecem inalterados. Apenas os valores absolutos dos raios são recalibrados, preservando as proporções definidas pela Lei de Murray, em que

$$r_j \propto Q_j^{1/\gamma}.$$

. Consequentemente, o volume dos vasos ($V \propto r^2$) e a resistência hidráulica ($R \propto r^{-4}$) são modificados, enquanto a estrutura geométrica da árvore permanece a mesma, tornando-a apenas visualmente mais espessa ou mais fina.

2. O raio da raiz aumenta com o número de terminais? Por quê?

Nos experimentos realizados ($R = 10$, $\gamma = 3$ e $M = 6$), variando-se o número de terminais entre 5 e 80, observou-se que o raio da raiz permaneceu praticamente constante, assumindo valores entre aproximadamente 0,0041 e 0,0042, sem apresentar tendência de crescimento com N_{term} . Esse comportamento é esperado nesta implementação, pois os parâmetros físicos Q_{perf} e ΔP permanecem constantes durante toda a execução. Dessa forma, a normalização força a resistência equivalente da árvore a assumir sempre o valor $[R_{\text{eq}} = \frac{\Delta P}{Q_{\text{perf}}}]$, independentemente da quantidade de terminais. Embora o aumento de N_{term} introduza mais segmentos na árvore, ele também aumenta o número de caminhos em paralelo, compensando parcialmente o crescimento da resistência provocado pelos novos ramos. Como resultado, o raio da raiz permanece praticamente constante. Caso Q_{perf} fosse proporcional ao número de terminais, representando uma demanda de fluxo fixa por terminal, seria esperado um aumento do raio da raiz, aproximadamente seguindo

$$r_{\text{raiz}} \propto Q_{\text{perf}}^{1/4}.$$

3. A otimização geométrica reduziu o volume intravascular?

Sim. Comparando uma mesma configuração ($N_{\text{term}} = 20$, $R = 10$, $\gamma = 3$ e $M = 6$), observou-se que a otimização geométrica reduziu o volume total da árvore de aproximadamente 0,00178 para valores entre 0,00147 e 0,00153, correspondendo a uma redução de aproximadamente 15% a 20%. Esse resultado confirma que permitir o deslocamento do ponto de bifurcação em uma vizinhança local, em vez de mantê-lo fixo no ponto médio do segmento, produz soluções com menor custo volumétrico.

4. Qual o impacto do parâmetro M na qualidade da solução?

Foram realizados experimentos utilizando $N_{term} = 20$ e

$$M \in \{2, 4, 6, 8, 12\},$$

considerando a média de três execuções para cada configuração. Observou-se que o volume total diminuiu de aproximadamente 0,00172 ($M = 2$) para um mínimo de cerca de 0,00140 ($M = 6$), apresentando pequeno aumento para valores maiores de M . Isso ocorre porque aumentar M significa avaliar um número maior de posições candidatas para a bifurcação, aumentando a probabilidade de encontrar uma solução de menor custo. Entretanto, como a busca permanece restrita a uma vizinhança local fixa (30% do comprimento do segmento), após determinado valor de M os novos pontos passam apenas a amostrar regiões muito próximas das já avaliadas, produzindo ganhos cada vez menores.

5. Qual o impacto do parâmetro M no tempo de execução?

O tempo de execução cresce de forma monotônica com o aumento de M . Nos experimentos com $N_{term} = 20$, o tempo médio aumentou de aproximadamente 0,29 s ($M = 2$) para 0,94 s ($M = 12$), representando um crescimento superior a três vezes. Esse comportamento é esperado, pois a busca geométrica avalia aproximadamente

$$\frac{(M+1)(M+2)}{2}$$

pontos candidatos, ou seja, um número proporcional a $O(M^2)$. Para cada ponto candidato é necessário recalcular toda a física da árvore, incluindo fluxos, raios, resistências e volume.

6. A função custo baseada em volume favorece árvores mais curtas, raios menores ou ambos?

A função objetivo utilizada é

$$J = \sum_j \pi r_j^2 l_j,$$

o que mostra que tanto o comprimento dos segmentos quanto seus respectivos raios contribuem para o custo final. Entretanto, durante a otimização geométrica, apenas a posição da bifurcação é modificada diretamente. Assim, o algoritmo atua principalmente sobre os comprimentos dos segmentos, enquanto os raios continuam sendo determinados pela distribuição de fluxo e pela normalização global de resistência. Desse modo, a otimização favorece principalmente árvores mais curtas. A redução observada nos raios ocorre apenas de forma indireta: uma árvore com menor comprimento apresenta menor resistência hidráulica, permitindo que a etapa de normalização alcance a resistência desejada utilizando raios ligeiramente menores.

7. Como a complexidade do algoritmo mudou em relação ao MiniCCO-0?

No MiniCCO-0, cada novo terminal era conectado diretamente ao segmento mais próximo, utilizando apenas critérios geométricos de distância. Dessa forma, não havia necessidade de recalcular a física da árvore nem realizar otimização da posição da bifurcação. No MiniCCO-1, para cada segmento candidato é executada uma busca geométrica composta por aproximadamente $O(M^2)$ posições candidatas. Para cada posição avaliada, toda a árvore precisa ser percorrida para atualizar fluxos, raios, resistências e volume, operação com custo aproximado de $O(N)$, onde N representa o número de segmentos. Como cada novo terminal ainda precisa testar aproximadamente K segmentos candidatos (com $K \approx N$), o custo por inserção torna-se aproximadamente

$$O(K \cdot M^2 \cdot N) \approx O(M^2 N^2),$$

enquanto no MiniCCO-0 o custo era aproximadamente $O(N)$.

Portanto, a otimização geométrica aumenta significativamente o custo computacional do algoritmo, porém produz árvores com menor volume e distribuição hidráulica mais adequada.

Análise de Complexidade

Considerando: N o número de segmentos da árvore, M a resolução da grade triangular e K o número de candidatos de conexão avaliados. No código entregue, `insereTerminal (tree.cpp)` avalia *todos* os K segmentos existentes como candidatos (não há restrição a vizinhos próximos, ao contrário do que o OpenCCO recomenda para evitar custo proibitivo), logo $K \approx N$ nesta implementação:

```
for (int idx = 0; idx < n; idx++) {                               // K = n candidatos avaliados
    ...
    Point X = otimizaBifurcacaoPorGrade(arvore, idx, A, B, C, M,
                                         Qterm, gamma, mu,
                                         domCentro, domR, distMinNos,
                                         &custo, &statsLocal);
    ...
}
```

Dentro de `otimizaBifurcacaoPorGrade`, para *cada* candidato, a grade percorre $O(M^2)$ pontos (aproximadamente $(M+1)(M+2)/2$, restritos a uma janela local desde a correção da bifurcação da raiz), e para *cada ponto* a árvore inteira é copiada e sua física é recalculada do zero:

```
for (int i = 0; i <= M; i++) {
    for (int j = 0; j <= M - i; j++) {                            // O(M^2) pontos candidatos
        ...
        Tree temp;
        temp.segmentos = arvore.segmentos;                        // copia O(N)
        ...
        inserirBifurcacao(temp, idxOld, X, C, &idBif, &idTerm);
        atualizaGeometriaFisica(temp, Qterm, gamma, mu);          // recalcula a arvore INTEIRA: O(N)
        ...
    }
}
```

Ou seja, a implementação **recalcula a árvore inteira a cada ponto candidato**, e não apenas os segmentos afetados pela bifurcação – não há nenhum tipo de atualização incremental. Isso confere ao custo de inserir um único terminal:

$$O\left(\underbrace{K}_{\text{candidatos}} \cdot \underbrace{M^2}_{\text{pontos da grade}} \cdot \underbrace{N}_{\text{recálculo da árvore por ponto}}\right)$$

que, como $K \approx N$ nesta implementação, se torna $O(M^2 N^2)$ por terminal inserido. Somando ao longo do crescimento da árvore inteira (de $N = 1$ até $N = 2N_{term} - 1$ segmentos), o custo total tende a $O\left(M^2 \sum_{i=1}^N i^2\right) = O(M^2 N^3)$.

Essa previsão cúbica em N é consistente com o que medimos experimentalmente ($R = 10$, $\gamma = 3$, $M = 6$, dobrando N_{term} sucessivamente):

N_{term}	5	10	20	40	80
Tempo (s)	0,022	0,092	0,448	2,487	15,969

A cada duplicação de N_{term} , o tempo multiplica por um fator que cresce de $4,2\times$ (de 5 para 10) até $6,4\times$ (de 40 para 80), ou seja, o expoente empírico de escala sobe de $\approx N^{2,1}$ para $\approx N^{2,7}$ conforme N cresce, se aproximando do $O(N^3)$ previsto teoricamente conforme os termos de ordem mais alta passam a dominar.

5 Considerações Finais

Neste trabalho foi desenvolvida uma implementação do algoritmo CCO (*Constrained Constructive Optimization*) para a geração de árvores arteriais em um domínio circular. A abordagem adotada permitiu construir estruturas ramificadas de forma incremental, conectando novos terminais à árvore existente

por meio de uma otimização geométrica local que minimiza o volume intravascular total, respeitando critérios de não interseção e de distância mínima entre nós. Os resultados obtidos demonstraram que o algoritmo é capaz de gerar árvores conectadas e bem distribuídas dentro do domínio, preservando características típicas de sistemas arteriais, como a presença de bifurcações, a conservação de fluxo entre segmentos e a expansão progressiva da estrutura à medida que novos terminais são inseridos.

Observou-se também que o aumento do número de terminais provoca um crescimento significativo da quantidade de candidatos avaliados e do tempo de execução, uma vez que cada nova conexão exige recalcular a física de toda a árvore. Além disso, a resolução da grade de otimização geométrica (parâmetro M) mostrou-se um fator relevante de compromisso entre qualidade da solução e custo computacional: valores maiores de M tendem a reduzir o volume final da árvore, mas aumentam consideravelmente o tempo de execução.